

Developer Action Guide: Implementing Inclusive and Accessible Terminology Scanners

This document serves as an operational runbook for an automated coding agent tasked with building or configuring a static repository scanner to enforce inclusive language, cognitive accessibility, and clear technical writing standards.

Task Objective

Develop and integrate rules into a repository scanner (utilizing tools like **Vale** and **Alex.js**, or custom scripts) to parse documentation, comments, and code configurations. The scanner must detect, flag, and suggest modifications for language issues that cause translation friction, cognitive overload, or non-inclusive environments.

Phase 1: Tooling Foundations

1. Vale Configuration (`.vale.ini`)

The scanner should use Vale to parse markdown, HTML, and text files. Configure Vale to leverage pre-built styles and load custom, repository-specific rules.

```
StylesPath = styles
MinAlertLevel = suggestion

[*.{md,html,txt}]
BasedOnStyles = Vale, Google, Microsoft, CustomInclusive

# Set up custom vocabulary to avoid flagging legal key terms in code blocks
Vocab = RepoTerms
```

2. Alex.js Configuration (`.alexrc`)

Configure Alex.js to catch gender-favoring, ableist, racist, and condescending language.

```
{
  "allow": [],
  "profanitySureness": 1,
  "git": true
}
```

Phase 2: Technical Rules & Token Replacement Catalog

Use the dictionaries below to build custom Vale VAML rules or Regex patterns inside the custom

Use the dictionaries below to build custom valid HTML rules or regex patterns inside the custom scanner logic.

Rule Category A: Technical Nomenclature & Legacy Terms

Source Framework: Google Developer Documentation Style Guide / ASWF / AOUSD

- **Logic:**
 - **In Markdown/Prose:** Flag legacy non-inclusive terms and suggest modern replacements.
 - **In Code/Configuration files:** If terms cannot be immediately deprecated, enforce strict code font formatting () or parenthetical introduction on first use, defaulting to the replacement in subsequent prose.
- **Replacements:**
 - `master / slave` → Suggest: `parent node / replica` (or context-appropriate pairs like `primary / secondary` , `controller / worker`)
 - `sanity check` → Suggest: `validation check` or `final check`
 - `cripples / cripples the service` → Suggest: `slows down the service` or `degrades the service`
 - `dummy` → Suggest: `placeholder` or `stub`

Rule Category B: Socially Charged, Ableist, Ageist, or Violent Metaphors

Source Framework: Google Style / ASWF / AOUSD / MDN

- **Logic:** Block terms that alienate disabled, neurodivergent, or global users, or utilize violent imagery.
- **Replacements:**
 - `normal / healthy` (when describing individuals without disabilities) → Suggest: `nondisabled` , `sighted` , `hearing` ,or `neurotypical`
 - `grandfathered / legacy` (as a verb/metaphorical process) → Suggest: `flagship` or `rollover`
 - `kill two birds with one stone` → Suggest: `solve two problems at once`
 - `wheelchair-bound / suffering from [disability]` → Suggest: `uses a wheelchair` or `experiencing [disability]` (Deficit-based to Asset-based transition)

Rule Category C: Diminishing Language & Adverbs of Assumption

Source Framework: Microsoft Writing Style Guide / 18F / plainlanguage.gov

- **Logic:** Words that assume task ease construct an "illusion of ease" and cause cognitive friction or user anxiety. Flag and eliminate.
- **Regex Check:**

```
\b(just|simply|obviously|clearly|basically|easy|easily|quite|very|quickly|effectively)\b
```
- **Prose Correction Pattern:**

• *Non-compliant:* "Simply configure the file."

- *Non-Compliant:* Simply configure the file.
- *Compliant (Direct Imperative):* "Configure the file."
- *Action:* Strip the adverb and capitalize the following verb.

Rule Category D: Structural, Cognitive & Localization Obstacles

Source Framework: WCAG 2.2 / COGA Task Force / Microsoft / Mailchimp

- **Rule D1: Modifier Stacks**
 - *Definition:* Chains of multiple modifying words preceding a noun (e.g., "enterprise production system database cluster configuration settings").
 - *Scanner Action:* Flag noun-modifying clusters exceeding 3 consecutive modifiers. Suggest breaking them up using prepositions (e.g., "settings for configuring database clusters in enterprise production systems").
- **Rule D2: Slash Notation / "and/or"**
 - *Definition:* Forward-slash notations that cause screen-reader hurdles and disrupt dyslexic or ADHD readers.
 - *Regex Check:* `\b\w+\s+\w+\b` (specifically target `and/or`)
 - *Scanner Action:* Suggest replacing with "or," "and," or restructuring the list.
- **Rule D3: Translation Fillers / Redundancies**
 - *Definition:* Words that inflate sentence length, confuse machine translation, and exclude non-native speakers.
 - *Replacements:*
 - `actually` → Omit
 - `absolutely` → Omit
 - `could possibly` → Suggest: `could`
 - `assemble together` → Suggest: `assemble`
- **Rule D4: Paragraph Complexity (Scannability)**
 - *Logic:* Readability degrades exponentially with excessive text. Limit cognitive load.
 - *Scanner Action:*
 - Flag any paragraph exceeding **5 sentences**.
 - Flag sentences that exceed a high-complexity syllable/word count limit (target plain language readability).
- **Rule D5: Paternalistic Phrasing**
 - *Logic:* Transition from deficit/paternalistic framing to neutral agency.
 - *Replacement:*
 - `empower` → Flag. Suggest restructuring around providing tools/resources that let the user complete actions directly.

Phase 3: Automated Enforcement Logic

I use the following Python class template to script custom checks inside the scanner engine. This

Use the following Python class template to script custom checks inside the scanner engine. This script can be wrapped as a custom GitHub Action or git pre-commit hook.

```
import re
import sys

class RepositoryScanner:
    def __init__(self):
        # Category C: Diminishing Language
        self.diminishing_regex = re.compile(
            r'\b(just|simply|obviously|clearly|basically|easy|easily|quite|very|q
            re.IGNORECASE
        )
        # Category D2: Slash Notation
        self.slash_regex = re.compile(r'\b\w+\ /\w+\b')

        # Token Mappings (Categories A & B)
        self.term_replacements = {
            r'\bmaster\b': 'parent node / controller',
            r'\bslave\b': 'replica / worker',
            r'\bsanity check\b': 'validation check / final check',
            r'\bscribble\b': 'slows down / degrades',
            r'\bdummy\b': 'placeholder / stub',
            r'\bgrandfathered\b': 'flagship / rollover',
            r'\bkill two birds with one stone\b': 'solve two problems at once',
            r'\bwheelchair-bound\b': 'uses a wheelchair',
            r'\bsuffering from a disability\b': 'experiencing a disability',
            r'\bempower\b': '[Avoid paternalistic language; focus on user agency]
        }

    def scan_text(self, text, filepath):
        errors = []
        lines = text.split('\n')

        # Paragraph length check tracking (Rule D4)
        current_paragraph_len = 0
        paragraph_start_line = 0

        for idx, line in enumerate(lines):
            line_num = idx + 1

            # Skip code blocks in markdown files
            if line.strip().startswith('```') or line.strip().startswith('``'):
                continue

            # 1. Test Diminishing Language
            for match in self.diminishing_regex.finditer(line):
                errors.append({
                    'file': filepath,
                    'line': line_num,
                    'rule': 'DiminishingLanguage',
                    'text': match.group(),
                    'fix': 'Remove or replace with a direct, active imperative ve
                })

            # 2. Test Slash Notations
```

```

    # 2. Test Slash Notations
    for match in self.slash_regex.finditer(line):
        errors.append({
            'file': filepath,
            'line': line_num,
            'rule': 'SlashNotation',
            'text': match.group(),
            'fix': "Avoid '/' or 'and/or'. Choose a definitive option or
        })

# 3. Test Static Term Replacements
for pattern, replacement in self.term_replacements.items():
    if re.search(pattern, line, re.IGNORECASE):
        errors.append({
            'file': filepath,
            'line': line_num,
            'rule': 'InclusivityViolation',
            'text': re.search(pattern, line, re.IGNORECASE).group(),
            'fix': f"Replace with: {replacement}"
        })

# 4. Sentence and Paragraph tracking (Rule D4)
if line.strip() == "":
    if current_paragraph_len > 5:
        errors.append({
            'file': filepath,
            'line': paragraph_start_line,
            'rule': 'ParagraphLengthLimit',
            'text': f"Paragraph spans {current_paragraph_len} sentences",
            'fix': "Break paragraph into smaller units of 5 sentences
        })
        current_paragraph_len = 0
    else:
        if current_paragraph_len == 0:
            paragraph_start_line = line_num
        sentences = [s for s in re.split(r'[.!?]+' , line) if s.strip()]
        current_paragraph_len += len(sentences)

return errors

# Usage inside scan workflows:
# scanner = RepositoryScanner()
# violations = scanner.scan_text(markdown_file_content, "docs/README.md")

```

Phase 4: CI/CD Pipeline Enforcement

To treat documentation with engineering rigor, configure the repository pipeline (e.g., GitHub Actions, GitLab CI) to execute Vale, Alex.js, and the custom RepositoryScanner class.

Recommended GitHub Action Workflow (`.github/workflows/docs-lint.yml`)

```

name: Prose and Inclusive Language Linting

on:
  pull_request:
    paths:
      - '**/*.md'
      - '**/*.txt'
      - 'docs/**'

jobs:
  lint-docs:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js (for Alex.js)
        uses: actions/setup-node@v4
        with:
          node-size: '20'

      - name: Run Alex.js
        run: |
          npm install -g alex
          alex . --fail

      - name: Run Vale Linter
        uses: errata-ai/vale-action@v2
        with:
          files: 'docs'
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

```

Phase 5: Verification & Testing Suite

For self-testing the scanner logic, verify that the system flags the following text sample containing multiple structural, cognitive, and linguistic errors:

To simply make a change, you must use our master/slave model. Obviously, everyone knows this is very easy and quickly empowers our disabled or wheelchair-bound users suffering from mobility issues. You can check the database config setting parameters and/or debug files to sanity check the server, which actually prevents it from being completely crippled by errors.

Expected Flagged Violations:

1. simply → Diminishing Language
2. master/slave → Non-inclusive nomenclature

3. Obviously, everyone knows this → Condescending assumption of cognitive ease

3. obviously, everyone knows this → Condescending assumption of cognitive ease
4. very easy, quickly → Unnecessary adverbs
5. empowers → Paternalistic phrasing
6. wheelchair-bound, suffering from → Deficit-based phrasing
7. database config setting parameters → Modifier stack (4 nouns modifying parameters)
8. and/or → Slash notation hazard
9. sanity check → Ableist phrase